

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1403

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I Pooja sree.M (192521403) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
```

```

}

int *ptr;
printf("%d\n", *ptr);

int total = 100;
int students = 0;

float average = total / students;

int i = 1;

while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks:

- (a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

Line	Error Type	Reason
11	Syntax	Missing semicolon (;) after char name[20] = "Compiler"
15	Semantic	findLength() called without passing the required argument (name)
17	Syntax	Missing closing parenthesis) in if(count > 5)
25	Runtime	marks[4] is out of bounds (valid indices: 0–3)
33	Syntax	Missing colon (:) after case 2
42	Runtime	Dereferencing an uninitialized pointer (*ptr)
47	Runtime	Division by zero (total / students)
53	Logical	Infinite loop because i is never incremented

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

Error	Category
Missing semicolon	Syntax Error
Missing argument in findLength()	Semantic Error
Missing) in if	Syntax Error
Array index out of bounds (marks[4])	Runtime Error
Missing : after case 2	Syntax Error
Uninitialized pointer dereference	Runtime Error
Division by zero	Runtime Error
Infinite loop	Logical Error

Lexical Error: None in the given program.

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

Error Category	Detected During
Lexical Error	Lexical Analysis
Syntax Error	Syntax Analysis (Parsing)
Semantic Error	Semantic Analysis
Logical Error	Not detected by compiler; found during testing/execution
Runtime Error	During program execution

Detected only during execution:

- Array index out of bounds
- Uninitialized pointer dereference
- Division by zero
- Infinite loop

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

CORRECTED CODE:

```
#include <stdio.h>
#include <string.h>
```

```
int findLength(char str[])
{
    return strlen(str);
}
```

```
int main()
```

```
{
    char name[20] = "Compiler";

    int count;
    count = findLength(name);

    if (count > 5)
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    // Print the last valid element
    printf("Marks = %d\n", marks[3]);

    int choice = 2;

    switch (choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2:
            printf("Option 2\n");
            break;

        default:
            printf("Invalid Choice\n");
    }

    int value = 50;
    int *ptr = &value;
    printf("Pointer Value = %d\n", *ptr);

    int total = 100;
    int students = 5;

    float average = (float)total / students;
    printf("Average = %.2f\n", average);

    int i = 1;
```

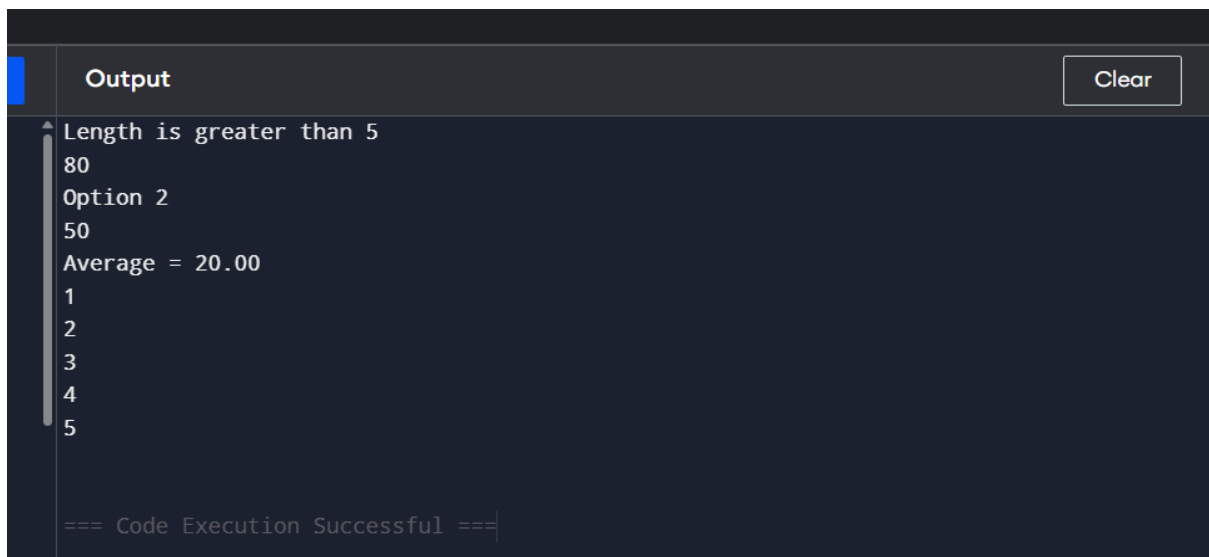
```

while (i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

OUTPUT:



```

Output
Clear

Length is greater than 5
80
Option 2
50
Average = 20.00
1
2
3
4
5

=== Code Execution Successful ===

```

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

- ❖ **Wall** – Enables common compiler warnings.
- ❖ **Wextra** – Detects additional possible programming mistakes.
- ❖ **Wpedantic** (or static analysis tools like clang-tidy) – Detects non-standard code and potential bugs before execution.

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}
```

```
int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n");
    }

    return 0;
```

Tasks:

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

Line No.	Syntax Error	Reason
3	int maximum(int x, int y	Missing closing parenthesis) in function definition.
18	printf("Maximum is 80\n")	Missing semicolon (;).
21	case 90	Missing colon (:) after case 90.
32	while(max > 75);	Missing closing brace } before while in the do-while loop.
35	printf("Limit Reached\n"	Missing closing parenthesis) and semicolon (;).
36	}	Extra closing brace due to previous syntax errors.
38	return 0;	main() function is not properly closed because the final closing brace } is missing.

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

Syntax Error	Category
Missing) in function definition	Missing Delimiter
Missing ; after printf()	Missing Statement Terminator
Missing : after case 90	Incorrect Switch Syntax
Missing } before while	Unmatched Braces
Missing) and ; in printf()	Incomplete Function Call
Missing final } for main()	Unmatched Braces

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

CORRECTED CODE:

```
#include <stdio.h>
```



```

int maximum(int x, int y)
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n");
            break;

        case 90:
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    }
    while(max > 75);

    if(max == 75)
        printf("Limit Reached\n");

    return 0;
}

```

OUTPUT:

```
Output
^ Maximum is 80
80
79
78
77
76
Limit Reached

=== Code Execution Successful ===
```

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

Parser Detection:

- The parser checks whether the program follows the grammar rules of C.
- If it encounters missing symbols (such as ;,), }, or :), it reports a syntax error.

Error Recovery:

- The parser skips incorrect tokens or inserts expected tokens.
- It continues parsing the remaining program so that multiple syntax errors can be reported in a single compilation instead of stopping at the first error.

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

- ❖ **Display the exact line number and the expected token** (e.g., *"Expected ';' after printf statement at line 18"*).
- ❖ **Suggest possible fixes** (e.g., *"Did you forget to add ':' after case 90?"*).

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int factorial(int n)
{
    if(n <= 1)
        return 1;

    return n * factorial(n - 1);
}

int main()
{
    int @num = 5;

    int value = 10

    int result;

    result = factorial();

    if(result >= 100)
    {
        printf("Large Number\n");
    }

    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120
            printf("Factorial of 5\n");
            break;

        default:
            printf("Other Value\n");
    }

    int numbers[5] = {2,4,6,8,10};
    printf("%d\n", numbers[5]);

    int *ptr;
    printf("%d\n", *ptr);

    int a = 50;

```

```

int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks:

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

Line No.	Error	Error Type	Reason
13	int @num = 5;	Lexical Error	@ is not a valid character in a C identifier.
15	int value = 10	Syntax Error	Missing semicolon (;).
19	result = factorial();	Semantic Error	factorial() requires one argument (num).
31	case 120	Syntax Error	Missing colon (:) after case 120.
39	printf("%d\n", numbers[5]);	Runtime Error	Array index 5 is out of bounds. Valid indices are 0–4.
42	printf("%d\n", *ptr);	Runtime Error	Pointer ptr is uninitialized. Dereferencing causes undefined behavior.
46	int c = a / b;	Runtime Error	Division by zero (b = 0).
48	int sum = result + 0 + value;	Code Optimization	Adding 0 is unnecessary. Compiler optimizes it to result + value.

51–54	while(i <= 5)	Logical / Runtime Error	i is never incremented, resulting in an infinite loop.
-------	---------------	-------------------------	--

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

Compiler Phase	Error
Lexical Analysis	Invalid identifier @num
Syntax Analysis	Missing ;, Missing : after case 120
Semantic Analysis	factorial() called without required argument
Intermediate Code Generation	Cannot generate correct intermediate code if syntax/semantic errors exist
Code Optimization	Expression result + 0 + value can be optimized to result + value (constant simplification)
Runtime	Array out of bounds, Uninitialized pointer, Division by zero, Infinite loop

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

Phase	Errors Detected
Lexical Analysis	Detects invalid token @ in variable name.
Syntax Analysis	Detects missing semicolon and missing colon in switch statement.
Semantic Analysis	Detects incorrect function call (factorial() without an argument).
Intermediate Code Generation	Generates intermediate representation only if no lexical, syntax, or semantic errors exist.
Code Optimization	Removes unnecessary computation such as adding 0 (result + 0 + value → result + value).
Runtime	Detects array index out of bounds, uninitialized pointer access, division by zero, and infinite loop during execution.

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

CORRECTED CODE:

```
#include <stdio.h>

int factorial(int n)
{
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
}

int main()
{
    int num = 5;

    int value = 10;

    int result;

    result = factorial(num);

    if (result >= 100)
    {
        printf("Large Number\n");
    }

    switch (result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120:
            printf("Factorial of 5\n");
            break;

        default:
            printf("Other Value\n");
    }
}
```

```

int numbers[5] = {2, 4, 6, 8, 10};
printf("%d\n", numbers[4]);

int val = 50;
int *ptr = &val;
printf("%d\n", *ptr);

int a = 50;
int b = 5;
int c = a / b;
printf("Division Result = %d\n", c);

int sum = result + value;
printf("Sum = %d\n", sum);

int i = 1;
while (i <= 5)
{
    printf("%d\n", i);
    i++;
}

return 0;
}

```

OUTPUT:

	Output
▲	<pre> Large Number Factorial of 5 10 50 Division Result = 10 Sum = 130 1 2 3 4 5 === Code Execution Successful === </pre>

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

- ❖ **Enable compiler warnings** (-Wall -Wextra) to detect issues such as missing function arguments, uninitialized variables, and suspicious code before execution.
- ❖ **Perform static analysis and optimization** to identify potential runtime problems (like division by zero and array bounds issues) and optimize unnecessary expressions (e.g., $\text{result} + 0 + \text{value} \rightarrow \text{result} + \text{value}$).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand